

Timeseries

How computers represent time and date?

POSIX Unix Epoch

```
from datetime import datetime, UTC  
import pytz
```

```
datetime.now()
```

```
datetime.datetime(2025, 5, 14, 11, 17, 38, 483490)
```

```
datetime.now(tz=UTC)
```

```
datetime.datetime(2025, 5, 14, 9, 17, 38, 528957, tzinfo=datetime.timezone.utc)
```

POSIX Unix Epoch

```
# Seconds since "epoch" where epoch is 01.01.1970 00:00 UTC on linux/unix/mac/android  
datetime.now().timestamp()
```

1747214258.581897

```
# This of course does not care about timestamp (which is a good thing)  
datetime.now(tz=UTC).timestamp()
```

1747214258.629198

MS Ecosystem

Idea: Days since 1.Jan.1990

1.0 = 1 Day

1 hour = $1/24 \sim 0.04166$

MS Ecosystem

Practice:

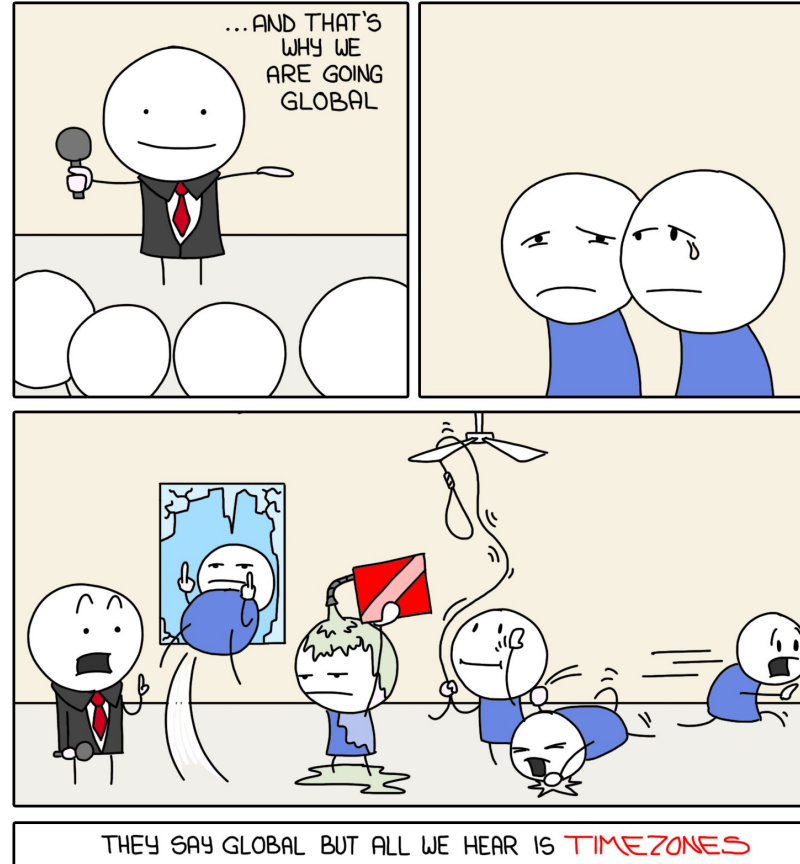
- Bug in Lotus 1-2-3
- 1900 was considered as leap year
- VB Script tries to fix dates before 1.March 1900
- Excel just rolls with it

C1		<i>f_x</i>	=vba_date(A1)
	A	B	C
1	0	01/00/1900	12/30/1899
2	1	01/01/1900	12/31/1899
3	2	01/02/1900	01/01/1900
4	3	01/03/1900	01/02/1900
5	4	01/04/1900	01/03/1900
6	5	01/05/1900	01/04/1900
7	6	01/06/1900	01/05/1900
8	7	01/07/1900	01/06/1900
9	8	01/08/1900	01/07/1900
10	9	01/09/1900	01/08/1900
11	10	01/10/1900	01/09/1900
12	53	02/22/1900	02/21/1900
13	54	02/23/1900	02/22/1900
14	55	02/24/1900	02/23/1900
15	56	02/25/1900	02/24/1900
16	57	02/26/1900	02/25/1900
17	58	02/27/1900	02/26/1900
18	59	02/28/1900	02/27/1900
19	60	02/29/1900	02/28/1900
20	61	03/01/1900	03/01/1900
21	62	03/02/1900	03/02/1900
22	63	03/03/1900	03/03/1900
23	64	03/04/1900	03/04/1900
24	65	03/05/1900	03/05/1900

- https://learn.microsoft.com/en-us/archive/blogs/ericlippert/erics-complete-guide-to-vt_date
- <https://stackoverflow.com/questions/36378476/why-does-the-date-returns-31-12-1899-when-1-is-passed-to-it#36378997>
- <https://www.joelonsoftware.com/2006/06/16/my-first-billg-review/>

Time zones

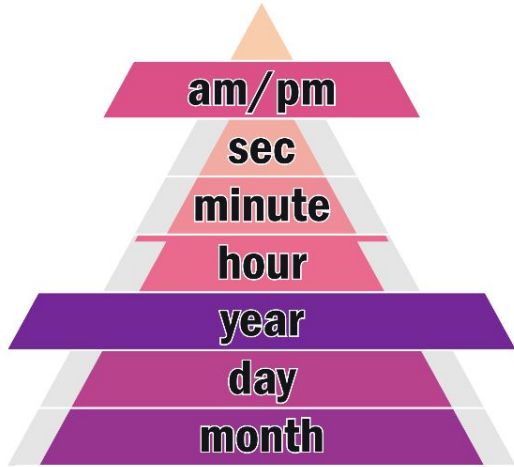
GOING GLOBAL



MONKEYUSER.COM

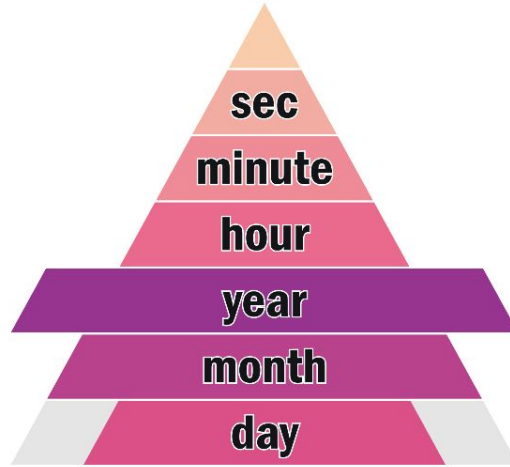
Time formats

Time formats



American

12/31/2021 11:59:59 PM



European

31.12.2021 23:59:59



International

2021-12-31T23:59:59

Sources: CLDR 38.1 / ISO 8601

ISO 8601

```
# Local time, creates time in the timezone set on your computer  
dt_iso_1 = '2025-05-14T10:30:09'
```

```
datetime.fromisoformat(dt_iso_1)
```

```
datetime.datetime(2025, 5, 14, 10, 30, 9)
```

```
# With timezone information  
dt_iso_utc_1 = '2025-05-14T10:30:09.000Z'
```

```
datetime.fromisoformat(dt_iso_utc_1)
```

```
datetime.datetime(2025, 5, 14, 10, 30, 9, tzinfo=datetime.timezone.utc)
```

```
datetime.fromisoformat(dt_iso_utc_2)
```

```
datetime.datetime(2025, 5, 14, 10, 30, 9, tzinfo=datetime.timezone.utc)
```

Take away message

- Avoid dealing with time zones at all cost
 - Avoid time zones with daylight saving time like the pest
 - Always make TZ aware, and use only UTC
 - Be careful when you are provided data that Excel or VBS touched
 - Strictly stick to ISO 8601
 - If you need precision and non-ambiguity down to the second posix unix epoch is also not for you
-
- <https://www.youtube.com/watch?v=-5wpm-gesOY>

Performing calculations on datetime

```
from datetime import datetime, timedelta
```

```
dt1 = datetime.fromisoformat('2025-05-14T10:30:09Z')  
dt2 = datetime.fromisoformat('2025-05-12T21:12:33Z')
```

```
dt1 - dt2
```

```
datetime.timedelta(days=1, seconds=47856)
```

```
td = timedelta(hours=5)  
dt1 + td
```

```
datetime.datetime(2025, 5, 14, 15, 30, 9, tzinfo=datetime.timezone.utc)
```

Reading timeseries in pandas

```
tawes_utc = """
time,station,cglo_j,rr,tl_mittel,vv_mittel,p_mittel,tlmin,tlmax
1999-04-09T00:00+00:00,13305,, -1,7.4,1.0,962.1,2.2,12.6
1999-04-10T00:00+00:00,13305,, -1,8.3,1.0,957.7,-0.6,17.1
1999-04-11T00:00+00:00,13305,, -1,10.9,1.5,951.9,7.5,14.3
1999-04-12T00:00+00:00,13305,,2.5,8.1,1.0,947.4,4.5,11.7
1999-04-13T00:00+00:00,13305,,7.8,9.1,0.6,939.7,6.2,12.0
"""
```

```
df = pandas.read_csv(StringIO(tawes_utc))
df.head(5)
```

	time	station	cglo_j	rr	tl_mittel	vv_mittel	p_mittel	tlmin	tlmax
0	1999-04-09T00:00+00:00	13305	NaN	-1.0	7.4	1.0	962.1	2.2	12.6
1	1999-04-10T00:00+00:00	13305	NaN	-1.0	8.3	1.0	957.7	-0.6	17.1
2	1999-04-11T00:00+00:00	13305	NaN	-1.0	10.9	1.5	951.9	7.5	14.3

Reading timeseries in pandas

```
df['time'].dtype
```

```
dtype('O')
```

```
df['time'] = pandas.to_datetime(df['time'])  
df['time'].dtype
```

```
datetime64[ns, UTC]
```

```
df.head(5)
```

	time	station	cglo_j	rr	tl_mittel	vv_mittel	p_mittel	tlmin	tlmax
0	1999-04-09 00:00:00+00:00	13305	NaN	-1.0	7.4	1.0	962.1	2.2	12.6
1	1999-04-10 00:00:00+00:00	13305	NaN	-1.0	8.3	1.0	957.7	-0.6	17.1
2	1999-04-11 00:00:00+00:00	13305	NaN	-1.0	10.9	1.5	951.9	7.5	14.3
3	1999-04-12 00:00:00+00:00	13305	NaN	-1.0	8.1	1.0	947.4	4.5	11.7

Pandas timeseries index

```
df = df.set_index('time')  
df.head(5)
```

	station	cglo_j	rr	tl_mittel	vv_mittel	p_mittel	tlmin	tlmax
time								
1999-04-09 00:00:00+00:00	13305	NaN	-1.0	7.4	1.0	962.1	2.2	12.6
1999-04-10 00:00:00+00:00	13305	NaN	-1.0	8.3	1.0	957.7	-0.6	17.1
1999-04-11 00:00:00+00:00	13305	NaN	-1.0	10.9	1.5	951.9	7.5	14.3
1999-04-12 00:00:00+00:00	13305	NaN	2.5	8.1	1.0	947.4	4.5	11.7
1999-04-13 00:00:00+00:00	13305	NaN	7.8	9.1	0.6	939.7	6.2	12.0

Indexing

```
df['1999-04-09 00:00:00' : '1999-04-10 00:00:00']
```

	station	cglo_j	rr	tl_mittel	vv_mittel	p_mittel	tmin	tmax
time								
1999-04-09 00:00:00+00:00	13305	NaN	-1.0	7.4	1.0	962.1	2.2	12.6
1999-04-10 00:00:00+00:00	13305	NaN	-1.0	8.3	1.0	957.7	-0.6	17.1

```
df['1999-04-09': '1999-04-10']
```

	station	cglo_j	rr	tl_mittel	vv_mittel	p_mittel	tmin	tmax
time								
1999-04-09 00:00:00+00:00	13305	NaN	-1.0	7.4	1.0	962.1	2.2	12.6
1999-04-10 00:00:00+00:00	13305	NaN	-1.0	8.3	1.0	957.7	-0.6	17.1

```
df[datetime(year=1999, month=4, day=1, tzinfo=UTC): datetime(year=1999, month=4, day=10, tzinfo=UTC)]
```

	station	cglo_j	rr	tl_mittel	vv_mittel	p_mittel	tmin	tmax
time								

Resampling

```
# resample to yearly averages  
df.resample('1YE').mean()
```

	station	cglo_j	rr	tl_mittel	vv_mittel	p_mittel	tlmin	tlmax
time								
2002-12-31 00:00:00+00:00	13305.0	1167.607735	1.861602	10.147238	1.174309	957.626796	5.231215	15.024033
2003-12-31 00:00:00+00:00	13305.0	1215.260989	1.024451	9.700000	1.173352	959.303846	4.046429	15.317033
2004-12-31 00:00:00+00:00	13305.0	1083.517808	1.616438	9.131781	1.126849	958.072603	4.322740	13.909041
2005-12-31 00:00:00+00:00	13305.0	1179.101928	2.275482	9.016253	1.205510	959.053994	4.079339	13.922590
2006-12-31 00:00:00+00:00	13305.0	1191.417582	1.362363	9.501374	1.218132	959.662088	4.452473	14.515659

Resampling

```
# Monthly max  
df.resample('1ME').max()
```

	station	cglo_j	rr	tl_mittel	vv_mittel	p_mittel	tlmin	tlmax
time								
2002-01-31 00:00:00+00:00	13305	855.0	5.4	5.5	2.1	977.3	2.0	12.6
2002-02-28 00:00:00+00:00	13305	1288.0	6.8	9.2	2.6	974.8	4.1	18.3

```
# Monthly median  
df.resample('1ME').median()
```

	station	cglo_j	rr	tl_mittel	vv_mittel	p_mittel	tlmin	tlmax
time								
2002-01-31 00:00:00+00:00	13305.0	486.0	-1.0	-3.80	1.0	965.60	-7.40	0.90
2002-02-28 00:00:00+00:00	13305.0	681.0	0.0	4.10	1.0	957.50	-0.30	8.70

Upsampling

```
df.resample('1h').mean()
```

	station	cglo_j	rr	tl_mittel	vv_mittel	p_mittel	tlmin	tlmax
time								
2002-01-01 00:00:00+00:00	13305.0	322.0	-1.0	-4.6	1.5	972.4	-9.4	0.2
2002-01-01 01:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2002-01-01 02:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2002-01-01 03:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2002-01-01 04:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
...	...	...	...	...	...	...	...	...
2018-05-20 20:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2018-05-20 21:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2018-05-20 22:00:00+00:00	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

Upsampling

```
df.resample('1h').ffill()
```

[illegible]

Upsampling

```
df.resample('1h').interpolate()
```

[illegible]